

# GraphTrust: Explainable Whole-Identity Graph Analysis for Detecting and Reducing Multi-Hop IAM Risk

Baktiyar Ablaihan

Nazarbayev Intellectual School, Kazakhstan

**Track:** AI, Data Science & Computing    **Code:** [github.com/sronters/graph](https://github.com/sronters/graph)

**Abstract.** Modern identity and access management (IAM) exposure can emerge from combinations of individually ordinary relationships among users, groups, roles, workloads, pipelines, policies, and cloud resources. This study asks whether whole-identity graph analysis detects and explains modeled multi-hop exposure more effectively than direct-permission and privileged-identity audits. GraphTrust compiles provider-normalized IAM evidence into a directed heterogeneous capability graph, performs bounded path analysis, produces an auditable Zero Trust Risk Index (ZTRI), and recommends raw permission changes using cost-aware graph optimization. We evaluate five detection methods on SEIB-2026, a deterministic synthetic-enterprise benchmark with three organizational profiles, paired clean and injected variants, fixed final seeds, hard negatives, and truth withheld until inference completes. Primary endpoints are risky-starting-identity recall, scenario recall, exact-path recall, ranking quality, false-positive burden, runtime, and peak memory. GraphTrust substantially outperforms direct and privileged-only baselines for transitive exposure and improves ranking over an untyped reachability baseline, but it does not improve recall over that untyped baseline; therefore typed semantics provide prioritization and explanation rather than universal detection superiority. Counterfactual remediation verifies proposed changes after application and checks protected business workflows. Results support a bounded claim: whole-identity reachability closes important audit blind spots, while typed risk modeling makes the resulting path volume more reviewable. Synthetic structure, ordinal weights, static snapshots, and bounded search limit external validity.

## 1 Research question and significance

**Question.** To what extent does typed, whole-identity graph analysis improve detection, ranking, explanation, and remediation of multi-hop paths to critical cloud resources compared with four precisely defined IAM-audit baselines?

Zero Trust Architecture rejects implicit trust based on network location and requires access decisions to consider subjects, assets, resources, and policy context ([National Institute of Standards and Technology, 2020, 2023](#)). IAM reviews that stop at direct grants can miss a low-privilege identity whose group membership, role assumption, service impersonation, or deployment authority creates transitive reachability. Existing work already demonstrates graph-based attack-path and IAM analysis; Principal Mapper and BloodHound are prominent examples ([NCC Group, 2022](#); [SpecterOps, 2026](#)). GraphTrust therefore does not claim to invent IAM graphs. Its contribution is a platform-neutral, truth-separated benchmark and controlled comparison that combines typed path ranking, decomposed risk explanations, and workflow-constrained remediation.

We preregistered three hypotheses: (H1) whole-identity analysis increases recall of planted transitive scenarios over direct and privileged-only audits; (H2) typed semantics improve top-ranked relevance over untyped traversal; and (H3) cost-aware cuts reduce exposure with fewer or lower-cost raw IAM changes while preserving protected workflows.

## 2 Method

### 2.1 SEIB-2026 benchmark and experimental matrix

SEIB-2026 generates three enterprise profiles—SaaS scale-up, regulated finance, and global hybrid—rather than uniformly random graphs. Each graph contains departments, correlated group and role assignment, human and non-human identities, policies, repositories, pipelines, secrets, and multi-cloud resources. Twelve scenario families inject documented dangerous paths and hard negatives. Paired variants share the same clean base graph, so changes in detection are attributable to the injected condition rather than unrelated topology.

The preregistered design crosses 3 profiles, 2 scales, 5 held-out seeds, 3 variants, and 5 methods: 450 planned immutable runs. Confirmatory evidence is the complete 225-run small-scale half: 30 injected graph units for detection and 15 clean controls for unmatched-finding burden. Each run retains data/configuration hashes, revision, environment, outputs, timings, and checksums; truth is joined only after inference. Uncertainty uses 10,000 paired graph-level bootstrap resamples, Friedman tests, and Holm-corrected paired Wilcoxon follow-ups. Reachability is potential exposure, not proof of exploitation.

Small graphs use a 250,000-path cap. Local medium diagnostics at caps from 250,000 to 100 exposed impractical repeated source-target search cost and are excluded. A frozen bounded follow-up and three high-RAM, checkpointed Colab notebooks exist, but were not completed in authenticated Colab; no medium/large performance result is claimed.



**Figure 1:** A highest-ranked path from an immutable GraphTrust run. Each transition is reproducible from raw IAM evidence; identifiers are shortened only in the visual.

## 2.2 Graph semantics, scoring, and baselines

Raw IAM evidence is compiled into effective capability edges while retaining the semantic rule and source evidence identifiers. Conditions use three states (true, false, unknown); unknown conditions are retained conservatively and labeled. Paths are simple, bounded to eight transitions, and limited by explicit per-source and global expansion caps. For path  $p$  ending at asset  $t$ ,

$$R(p) = C(t) S(p) \exp \left[ - \sum_{e \in p} (-\log x_e + \tau_e + \kappa_e) \right],$$

where  $C$  is normalized asset criticality,  $S$  starting-control weakness,  $x_e$  relative exploitability, and  $\tau_e, \kappa_e$  are typed-transition and condition penalties. Values are ordinal ranking parameters, not breach probabilities. ZTRI combines critical reach, best path, path diversity, blast radius, and control weakness; every component remains visible.

Baselines are B0 direct (one-hop grants), B1 privileged-only (typed traversal from pre-labeled privileged starts), B2 untyped graph (unit-cost reachability), and B3 native scope (direct, group, and provider-native relationships). This avoids the inaccurate claim that commercial IAM products inspect only direct permissions. AWS Access Analyzer, for example, uses automated reasoning for supported resource and policy scopes (Amazon Web Services, 2026).

## 2.3 Counterfactual remediation

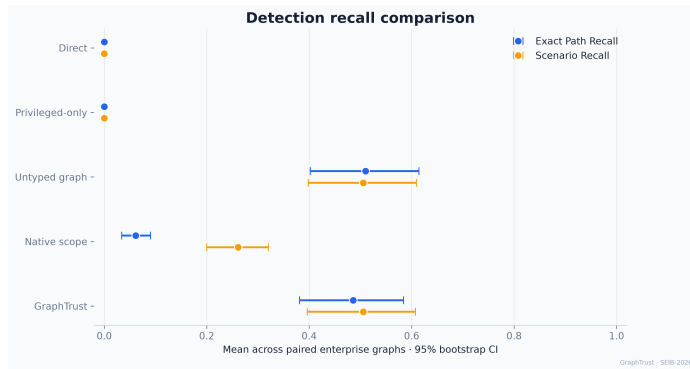
Removable raw relationships receive modeled business costs; mandatory edges and required workflows are protected. Four methods—degree greedy, risk greedy, weighted minimum cut, and iterative CP-SAT path hitting set—receive the same target. The verifier removes proposed raw edges, removes every effective transition supported by them, reruns bounded reachability, and then checks capability-specific protected workflows. A plan is labeled verified only if the specified blocked-path target and all workflow constraints hold. Recommendations never modify live permissions.

# 3 Results

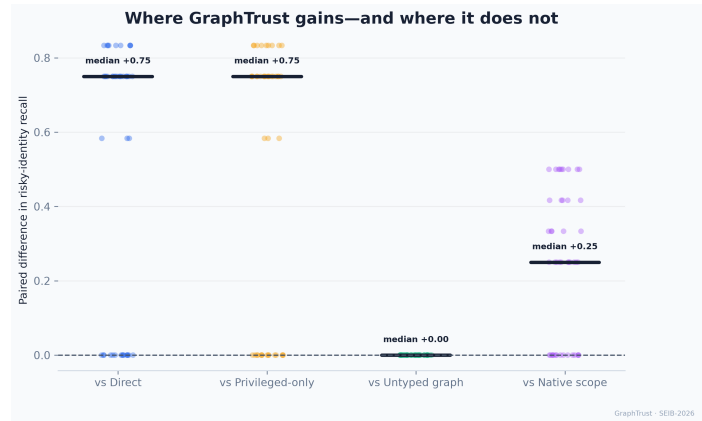
## 3.1 Detection and ranking

The final small-scale matrix contains 225 checksum-verified runs. On injected graphs, GraphTrust achieved mean risky-identity recall 0.758 and scenario recall 0.758, compared with 0.000/0.000 for direct audit, 0.000/0.000 for privileged-only audit, 0.392/0.392 for native scope, and 0.758/0.758 for untyped traversal. Thus H1 is supported against the narrow audit baselines but not against untyped whole-graph reachability.

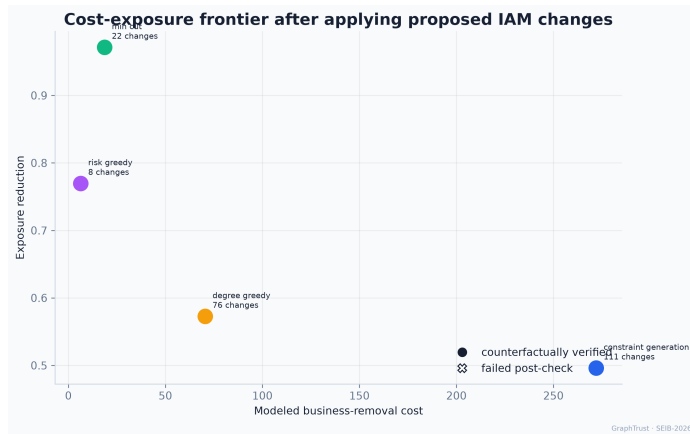
The more informative separation appears in ranking. GraphTrust reached NDCG@10 0.125, versus 0.002 for untyped traversal and 0.009 for native scope. This supports H2’s narrower claim: typed conditions and transition costs help surface relevant planted paths near the top even when raw reachability recall ties. GraphTrust exact-path recall was 0.729 versus 0.765 for untyped traversal; the paired difference was -0.036 (95% CI [-0.049, -0.023]). On clean controls, GraphTrust produced 13,786.9 unmatched paths per 1,000 identities, close to untyped traversal at 13,890.0. An unmatched synthetic path is not necessarily a real false alarm, but the volume shows why reachability must be ranked and reviewed rather than treated as an incident list.



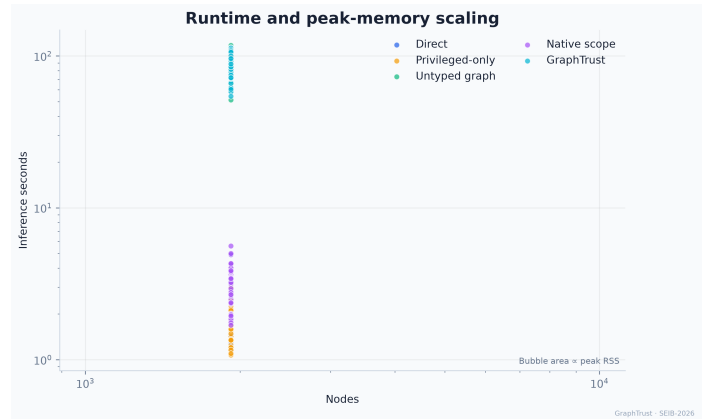
(a) Detection recall with bootstrap intervals.



(b) Graph-level paired differences.

**Figure 2:** Aggregate means alone can hide ties and profile-specific variation; paired results show exactly where GraphTrust gains.

(a) Verified remediation cost-exposure frontier.



(b) Runtime and peak resident memory.

**Figure 3:** Operational trade-offs are reported together with post-application verification and resource use.

Paired differences make the boundary explicit. Against untyped traversal, the risky-identity-recall difference was  $+0.000$  (95% bootstrap CI  $[+0.000, +0.000]$ ), while the NDCG@10 difference was  $+0.123$  (95% CI  $[+0.083, +0.164]$ ). The five-method Friedman tests gave  $p = 5.34e - 25$  for risky-identity recall and  $p = 3.37e - 13$  for NDCG@10. Inferential tests use the enterprise graph, not each reported path, as the independent paired unit.

### 3.2 Remediation and scalability

On the preregistered injected-mixed verification graph, weighted min-cut proposed 22 raw IAM changes at modeled cost 18.58. Counterfactual application reduced enumerated relative exposure by 0.971 and preserved all protected workflows; verification was yes. Greedy alternatives exposed different cost/change trade-offs, so “minimal” depends on the objective. A local large-data diagnostic also exposed and eliminated a recursion-dependent quality check; without authenticated Colab receipts, this remains an engineering diagnostic rather than scale evidence.

## 4 Limitations and threats to validity

Synthetic structure does not establish real-world prevalence. Injected truth may favor aligned methods, while unlabeled plausible paths lower measured precision. Weights, removal costs, asset labels, and protected workflows are generated assumptions. Provider normalization omits some policy exceptions, sessions, denials, and temporal conditions; bounded top- $K$  search can omit paths. Three profiles are not a representative company sample. Reachability is potential exposure, not intent, exploit success, or incident probability.

## 5 Conclusion and impact

In SEIB-2026, whole-identity reachability reveals transitive paths missed by direct and privileged-only audits. Typed semantics match untyped traversal’s risky-identity recall but improve top-ranked relevance, provenance, and explanation. Counterfactual remediation maps choke points to raw IAM changes and verifies exposure reduction plus workflow preservation. GraphTrust therefore complements provider-native IAM tools; it is not autonomous enforcement evidence.

## References

- Amazon Web Services. (2026). *IAM Access Analyzer concepts and findings*. <https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-concepts.html>
- SpecterOps. (2026). *BloodHound: Six degrees of domain admin*. <https://github.com/SpecterOps/BloodHound>
- Csárdi, G., & Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal, Complex Systems*, 1695.
- Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference* (pp. 11–15).
- NCC Group. (2022). *Principal Mapper (PMapper)*. <https://github.com/nccgroup/PMapper>
- National Institute of Standards and Technology. (2020). *Zero trust architecture* (Special Publication 800-207). <https://doi.org/10.6028/NIST.SP.800-207>
- National Institute of Standards and Technology. (2023). *A zero trust architecture model for access control in cloud-native applications in multi-cloud environments* (Special Publication 800-207A). <https://doi.org/10.6028/NIST.SP.800-207A>
- Perron, L., Didier, F., & Gay, S. (2023). The CP-SAT-LP solver. In *29th International Conference on Principles and Practice of Constraint Programming* (pp. 3:1–3:2). Schloss Dagstuhl. <https://doi.org/10.4230/LIPIcs.CP.2023.3>
- Sheyner, O., Haines, J., Jha, S., Lippmann, R., & Wing, J. M. (2002). Automated generation and analysis of attack graphs. In *Proceedings of the 2002 IEEE Symposium on Security and Privacy* (pp. 273–284). <https://doi.org/10.1109/SECPRI.2002.1004377>

## AI Use Transparency Statement

OpenAI Codex assisted with interpreting the project specification, implementing and reviewing code, diagnosing scale and verification defects, running local automated experiments, generating figures, and editing this brief. The author directed the research question and remains responsible for understanding the method, reviewing every claim, verifying citations, and submitting the work. Synthetic data are produced by deterministic repository code; no real company or personal IAM data were used. Numerical claims are generated only from checksum-verified run artifacts. AI was not used to fabricate citations, experimental outcomes, collaborators, or human review. Docker definitions and a public verification workflow were prepared, but no Docker engine was available for an actual image-build/startup test. The three large Colab notebooks were prepared but not executed; they record environment data and immutable receipts when run.

## Appendix A: Reproducibility and experiment matrix

The repository pins Python dependencies, provides Docker and Compose definitions, and stores three profile-specific Colab notebooks with high-RAM gates, resume checkpoints, and receipts. The complete design is  $3 \times 2 \times 5 \times 3 \times 5 = 450$  planned small/medium runs, plus three large profile launches at one held-out seed. The completed evidence set is 225/225 small runs; 225 medium runs and all three authenticated Colab receipts remain pending. Immutable run IDs derive from the dataset checksum, resolved configuration, git commit, and method. A traceability validator rejects missing or corrupt artifacts before report generation. Figures are exported as SVG and 300-dpi PNG and retain source run IDs in accompanying tables.